Final Project: An Interpreter

This Assignment is Due on Thursday, December 18th at 11:59 PM

For your final project, you will be implementing an interpreter for a small programming language. An interpreter is a program that reads in the text of a programming language, and then executes it directly (languages that are interpreted include python and racket, unlike C which is compiled).

Unlike the assignments, there is a large amount of provided code, and you will just be writing a portion.

Pip: A Small Programming Language

Pip is the name of the programming language you will be implementing/writing an interpreter for. It looks like something between bash (the shell language, if you are familiar) and C, but you can think of as working like a very small subset of python. To see what I mean, let's go over the features pip has with some example programs.

```
fn main()
  if 1 < 2 then
    print true;
  else
    print false;
  fi
  a = 0;
  while (a < 5) do
    print a;
    a = a + 1;
  end
  return true;
end
```

Like C, pip programs consist of a set of functions, with a special function `main()` which does not take any arguments and is run when a file is interpreted. Also like C, statements end with semi-colons. Unlike C or python though, we don't use whitespace or brackets to indicate blocks for while/if statements or functions, but rather extra keywords. In particular, we use `then` to indicate the beginning of the then-block (the statements that will be executed if the condition is true) in an if statement, else for the else-block, and `fi` to indicate the end of an if statement. Likewise, we use `do` and `end` to mark the body of a while loop, `fn` to indicate a function declaration and `end` to indicate the end of a function.

Pip also makes the distinction between "statements" (things you run for their effects, one after the other) and "expressions" (things that yield values), just like C. As a rule of thumb, things that you can assign to variables or pass as arguments to functions are expressions, and things you couldn't do that to but

still run are statements. Here are all of the types of expressions supported by pip (that you will implement):

1. Variables (hopefully clear)
2. Integer literals (i.e, integer constants that you write out like `1`, `2`, `42`,...)
3. Boolean literals (i.e `true` and `false`)
4. Binary Operations (i.e things like `1 + 1`, `true || false`, `(a - b) < 1`, ... which all take 2 arguments (one on the left, one on the right)).
5. Function Calls (which look like `foo(a,b,c)`, same as python/C).

In pip, all expressions evaluate to some sort of *value*: either an integer or a boolean. There are no `void` functions: a function not returning is considered to be an error.

Here are all of the types of Statements supported by pip:

1. Variable assignment (e.g `a = 1;`)
2. Print statements (e.g `print 1;`.) Note that print is a special keyword that doesn't require parentheses, and works on any expression.
3. If statements (e.g `if foo then bar fi`).
4. If-else statements (e.g `if foo then bar else baz fi`)
5. While statements (e.g `while foo do bar end`)
6. Return statements (e.g `return 1;`)
7. Expressions. This one is a little weird, but you can run an expression as a statement. We do this all the time in C when we call a function without caring about its return value.

Another key notion that will be relevant for you as you implement is that of *scoping*. The scope of a variable is the part of your program where it is valid. The place it is relevant in pip is functions. Take a look at the following pip program:

```
fn foo(a,b)
  c = a + b;
  return c;
end
fn main()
  x = 1
  y = 2
  print foo(x,y);
  return 0;
end
```

This is a perfectly valid program, because we respect the scoping of the variables: we only use things declared within the same function (as a variable or an argument). If we changed `foo(a,b)` to be like so:

```
fn foo(a,b)
  c = x + y;
  return c;
```

```
end
```

Then the program would not be valid, because `x` and `y` are not inside the scope of `foo` (even though they *are* in the scope of `main`). The *values* of `x` and `y` (1,2) become the values of a and b inside the function for the call, however, as we would expect from C.

How Interpreters Work: A Crash Course

As far as we are concerned for this project, there are 2 main phases of an interpreter: Parsing and Evaluation.

Parsing is the act of taking program text (the actual characters you write) and turning it into something called an *Abstract Syntax Tree* (AST). How Parsing actually works is outside of our concern for right now (feel free to talk to me about it :)), but we do need to understand ASTs. ASTs are structured ways of representing the text of a program. We don't really care about where the whitespace is, or where the semicolons are etc., but we do care about e.g the order in which we need to evaluate binary operations. We can capture this structure with a tree. As a toy example, imagine we have the expression `(3 * 2) + 1`. This is clearly different from `3 * (2 + 1)`, but how do we represent that? The answer is that we represent them as 2 different trees, that look like the following:

```
   +
  / \
 *   1
/ \
3  2

   *
  / \
 3   +
    / \
    2  1
```

respectively. This captures the order we care about really cleanly: all we have to do is recurse along the tree to get all the info we need to calculate the answer.

We can do this same trick for more complicated expressions and statements as well. say we have an if statement:

```
if (1 + 2) > 3 then
  print true;
else
  print false;
  return false;
fi
```

This would correspond to the following tree:

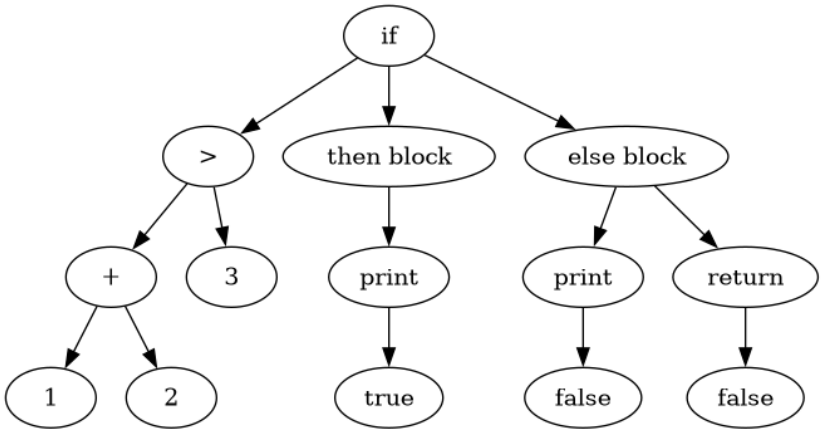

Figure 1: if syntax tree

With the left branch corresponding to the condition, the middle branch corresponding to the set of statements that will run if the condition is true, and the right branch corresponding to the set of statements that will run if it is false.

This tree is exactly what we need to actually evaluate the program. We want our interpreter to actually do stuff! We can do this by just "walking the tree": recurring along it, evaluating each branch of the AST node in the right way.

We've gone over very briefly the technique for expressions: for some expressions, it's clear what they should evaluate to without any work. Integers should evaluate to the integer, booleans should evaluate to the corresponding boolean, etc. Binary operations are only slightly more complex: each node has 2 further expressions that we can recursively evaluate, and then apply the corresponding operation to the values of each of the sides. To talk about variables and function calls, we should first talk about statements.

For statements, there is a somewhat different picture, because we are not trying to evaluate to some particular value, but instead we want to get some effect. When we evaluate a print statement, we want to see the expression on the screen, when we evaluate an assignment statement, we want to have the variable changed for the next expression, etc. Print statements aren't complex: when we evaluate a print statement, we just evaluate the thing we want to print, and then print it using the normal C `printf`. Assignments require a bit more machinery. If we want the next statement to be able to reference the new value for the variable, we need to pass that information along. We do this with map called the *enviroment*, which is some sort of mapping of names of variables to values. When an assignment statement `v = 1+1` is run, it evaluates the right hand side (`1+1`), and then stores that value in the environment with the name `v`, overwriting the existing value if there is one already. Then, when we evaluate the next statement, we do it with that updated environment.

Then, evaluating variables is a simple act of looking them up in the environment. Functions calls are a similar game: we want to just evaluate all of the statements in the body, but first we need to evaluate the arguments, and then add them to a new environment for the function with the right names. Note that each function gets its own environment to capture the scoping rules we discussed in the previous section There is one extra step: we also need to look up the function to find out the names of its arguments, and what its body is. As such we maintain a separate list of functions that we look up in when we evaluate a function call. There are a few other things we must handle (like how to evaluate if statements), but figuring these out will be part of your task.

Of course in real life the ASTs are not drawings of trees, but trees made of tagged unions and pointers.

Rundown of the Project Structure and Important Types

PROVIDED: Absyn.{c,h}, Buffer.{c,h}, Lexer.{c,h}, Parser.{c,h}, Printer.{c,h}, ...

These are all automatically generated files with the parser code. They're needed to get the AST you will work with, but you don't need to care about them directly.

PROVIDED: Ast.h

`Ast.h` contains the tagged unions that the AST consists of. Hopefully the names are clear enough to see how they correspond to the pictures I was drawing above. One thing to note is that any time there is an array (or a double pointer representing an array of pointers), you can assume that it is `NULL`-terminated

PROVIDED: Ast.c

`Ast.c` contains the methods for turning the output of the parser (which has some things we don't care about) into the AST. You will use the result of these functions, but shouldn't need to call them yourself, as that is part of the provided code.

PROVIDED: Main.c

`Main.c` contains the glue to actually read a program from a file and execute it.

PROVIDED: Eval.h

`[Eval.h](Eval.h)` Contains the definition of an environment (`env_t`) (which is a linked list of pairs of values and names), and a values (`val_t`), the declaration of the functions you will use to interact with the environment, and the declaration of the evaluation functions you will write (the definitions of which you will write in `Eval.c`).

`val_t` is a little strange looking so I will explain it here.

```
typedef struct {
  int val : 30;
  unsigned int is_bool : 1;
  unsigned int is_invalid : 1;
} val_t;
```

This is a bit field, with the number after the colon indicating how many bits the field takes up. The main value is a 30 bit integer, which can represent either an integer or a boolean. If the is_bool bit is set to 1, then we view `val` as a bool. For example, when we print a value, we want to print `true` and not `1` if we are printing a bool. The most important field is perhaps `is_invalid`. Most of the way we get `val_t`s can fail (like looking up a variable in an environment; what if it's not there?). The way we indicate failure is by returning a `val_t` where `.is_invalid` is set to 1. This means that **Every time we get a val_t from a function, we need to check this field to see if it is valid to use**. If it isn't, we need to bail and indicate failure. This unfortunately requires some extra though to make sure you are checking every time, but it's the same situation as with pointers and null. That's the nature of this language :). Why not used a tagged union? Because we want our interpreter to be really fast/memory efficient and we're going working with these a lot. Unfortunately the way tagged unions work with bitfields is a little strange (out of scope of this class), so we're going to do it this way.

TO FINISH: Eval.c

Here is where you will actually do the vast majority your work. Provided are the functions to interact with the environment. Your primary task for this assignment is to write 4 functions:

`eval_expr()` This function evaluates an expression, given a (pointer to an) expression, an environment, and a list of functions. If at any point in the evaluation (which should follow the pattern described in the previous section) an invalid value is given (e.g when evaluating a sub-expression or looking up a variable in the environment), computation should stop immediately and an invalid value should be returned.

There are 2 cases where you will need to print inside this function: if a variable cannot be found in the environment, or if an undefined function is called. In the first case, you should invoke the following:

```
printf("ERROR: Couldn't find variable %s in scope\n", ????);
```

(and then promptly return an invalid value), with the variable name supplied as the argument. In the second case you should invoke the following:

```
printf("ERROR: Tried to call invalid function %s\n",????);
```

with the invalid function's named supplied as the argument (returning as above).

`eval_statement()` This function evaluates a statement, and returns 1 if a return was evaluated when evaluating the statement, or if the function encountered an error (e.g by `eval_expr` being called inside and returning an invalid value). It returns 0 otherwise.

This function makes use of a pattern that is common in C but we have not seen before: that of an "out parameter." This describes an argument to a function which is a pointer, which is used solely to be written to by the function to pass a value back. We use this so we can indicate if our function has returned or encountered an error with the return value, and pass the particular value it returned with (if applicable) back by writing to this variable (in our case `ret_val`). All this means that when a return statement is encountered, we write the value of the expression being returned to `ret_val`, and then return `1`. Likewise, if we encounter an error (e.g by encountering an invalid value), we also return `1` to indicate computation should stop, but we don't write anything to `ret_val`. If the statement evaluates normally and doesn't return, we return `0` and don't write anything to `ret_val`.

How one should evaluate each statement should be somewhat clear from the previous section and the structure of `eval_expr` (what C construct would you use to implement the `if` statement evaluation correctly?). That said, if you struggle with what to do for some cases after some thought, don't hesitate to ask for help.

`eval_block()` This function works very similarly to `eval_statement`, but evaluates all of the statements in a null-terminated array of statements, returning `1` if any of the statements returns `1`, and `0` otherwise.

`eval_func()` This evaluates a function call. It takes in a pointer to a function (as a `fundecl_t*`), a null-terminated array of expressions that were given as arguments, the environment to evaluate those expressions with, and the list of defined functions. It creates a new empty environment for the function body (called, e.g `fn_env`), and then evaluates each of the passed in arguments (in `arg_expr_arr`, using `arg_env`) adding them to the new environment with the names of the arguments defined in the function (see the `args` field of fundecl_t). When evaluating each of the passed in arguments, make sure to check if they are invalid and return appropriately if so. Then create a `val_t` with `.is_invalid = 1` on the stack to serve as the out parameter for the next step. Then, evaluate the body of the function (using another function you've written), passing in the environment you just added the arguments to, and a pointer to the invalid `val_t`.

Then, clean anything up that needs to be free'd, and return. If the function does not return a value (how do you check that?) invoke the following:

```
printf("ERROR: Function %s did not return!\n",????);
```

with the function name, and return an invalid `val_t`.

What You Need To Do

You need to write the 4 functions specified in `Eval.c`. To test your work, you can use the autograder we will use for grading by running `make test`. To get started, copy the `project` folder in the C291-Materials directory to your repository. To Submit, commit and push your changes to github. Then, copy the hash (which you can see with `git log -1`), and submit that to canvas, just like the previous assignments.

Grading/Bonus Points

75% of your grade will come from passing the autograder tests (3 points per test). No partial credit will be given for failed tests. The rest will come from manual inspection, seeing both if your code seems to have some of the ideas corred (e.g if the tests fail), and for style, and comments, like previous assignments.

For an additional up to 10% of the grade: once you have the interpreter working, write a non-trivial program in pip, and execute it using the interpreter you wrote (a lot of). Points will be awarded based on being correct pip code, and how complex/interesting the program is. Because this is very open ended, there will be no autograder tests. You will need to make sure your program works (by writing the finished interpreter on it) and does something neat. Note that these are bonus points, and can only help your grade.

This Project is due Thursday, December 18th at 23:59 (11:59 PM).